

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB- ПРИЛОЖЕНИЙ

Лекция 11а

Объекты JavaScript

1. Объекты как ассоциативные массивы
2. Перебор свойств
3. Передача по ссылке
4. Методы объектов
5. Преобразование объектов: `toString` и `valueOf`
6. Создание объектов через `"new"`. Конструктор.
7. Дескрипторы, геттеры и сеттеры свойств

ОБЪЕКТЫ КАК АССОЦИАТИВНЫЕ МАССИВЫ

Ассоциативный массив – структура данных, в которой можно хранить любые данные в формате ключ-значение



`objectName.propertyName`

объекты в JavaScript могут иметь в качестве ключа только тип String

```
let myCar = new Object();  
myCar.make = "Ford";  
myCar.model = "Mustang";  
myCar.year = 1969;
```

```
myCar["make"] = "Ford";  
myCar["model"] = "Mustang";  
myCar["year"] = 1969;
```

СОЗДАНИЕ ОБЪЕКТОВ

1. `o = new Object();`
2. `o = {};` // пустые фигурные скобки или инициализатор объекта
// со списком свойств=значений
3. Функция конструктор
4. Метод `Object.create`

Операции с объектом

```
let person = {}; // пока пустой объект.свойство
```

```
// при присвоении свойства в объекте автоматически создаётся "ящик"
```

```
// с именем "name" и в него записывается содержимое 'Вася'
```

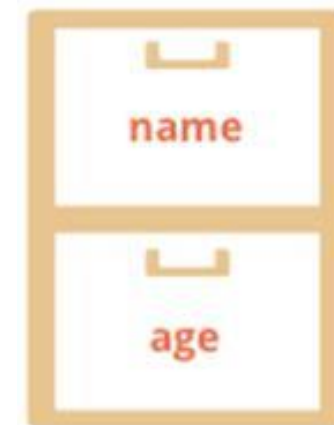
```
person.name = 'Вася';
```

```
// запишем ещё одно свойство:
```

```
//с именем 'age' и значением 25
```

```
person.age = 25;
```

```
alert( person.name + ': ' + person.age ); // "Вася: 25"
```



можно обратиться к любому свойству объекта, даже если его нет.

Ошибки не будет.

если свойство не существует, то вернется undefined:

```
let person = {};
```

```
alert( person.lalala ); // undefined, нет свойства с ключом lalala
```

```
let person = { name: "Василий"};
```

```
alert( person.lalala === undefined ); // true, свойства нет
```

```
alert( person.name === undefined ); // false, свойство есть.
```

ДОСТУП ЧЕРЕЗ КВАДРАТНЫЕ СКОБКИ

```
let myObj = new Object(),  
    str = "myString",  
    rand = Math.random(),  
    obj = new Object();  
myObj.type          = "Dot syntax";  
myObj["date created"] = "String with space";  
myObj[str]          = "String value";  
myObj[rand]         = "Random Number";  
myObj[obj]          = "Object";  
myObj[""]           = "Even an empty string";  
  
console.log(myObj);
```

```
[object Object] {  
  : "Even an empty string",  
  0.9527493129767106: "Random Number",  
  [object Object]: "Object",  
  date created: "String with space",  
  myString: "String value",  
  type: "Dot syntax"  
}
```

ДОСТУП ЧЕРЕЗ КВАДРАТНЫЕ СКОБКИ

объект['свойство']:

```
let person = { };
```

```
person['name'] = 'Вася'; // то же что и person.name = 'Вася'
```

```
person['любимый стиль музыки'] = 'Джаз';
```

```
person.любимый стиль музыки = 'Джаз'; // ошибка
```

```
alert(person['любимый стиль музыки']);
```

Вычисляемые свойства

```
let fruit = prompt("Какой фрукт купить?", "apple");
```

```
let bag = {  
  [fruit]: 5, // имя свойства будет взято из переменной fruit  
};
```

```
alert( bag.apple ); // 5, если fruit="apple"
```

Объект, объявленный как константа, может быть изменён

```
const myCar = new Object();
```

```
let propertyName = "make";  
myCar[propertyName] = "Ford";
```

```
propertyName = "model";  
myCar[propertyName] = "Mustang";
```

```
console.log(myCar);
```

```
myCar.color; // undefined
```

```
[object Object] {  
  make: "Ford",  
  model: "Mustang"  
}
```

объявление const защищает от изменений только саму переменную myCar , а не её содержимое.

Определение const выдаст ошибку только если мы присвоим переменной другое значение: myCar =

Удаление свойства

```
let person = {};
```

```
person.name = 'Вася';
```

```
person.age = 25;
```

```
delete person.age;
```



Проверка существования свойства

Оператор `in`: левый операнд – строка с именем свойства, правый операнд – объект.

Возвращает `true`, если объект имеет собственное или унаследованное свойство с указанным именем:

```
var o = { x: 1 };  
"x" in o;           // true  
"y" in o;           // false: o не имеет свойства y  
"toString" in o;    // true: o наследует свойство toString
```

Проверка существования свойства

Метод объекта `hasOwnProperty()` проверяет, имеет ли объект собственное свойство с указанным именем. Для наследуемых свойств он возвращает `false`:

```
var o = { x: 1 };  
o.hasOwnProperty("x");           // true  
o.hasOwnProperty("y");           // false  
o.hasOwnProperty("toString");    // false
```

Объявление со свойствами

Объект можно заполнить значениями при создании, указав их в фигурных скобках (литеральный синтаксис):

{ ключ1: значение1, ключ2: значение2, ... }

```
let menuSetup = { width: 300, height: 200, title: "Menu"};
```

// то же самое, что:

```
let menuSetup = {};
```

```
menuSetup.width = 300;
```

```
menuSetup.height = 200;
```

```
menuSetup.title = 'Menu';
```

В качестве значения можно тут же указать и другой объект:

```
let user = {  
  name: "Таня",  
  age: 25,  
  size: {  
    top: 90,  
    middle: 60,  
    bottom: 90  
  }  
}  
  
alert(user.name); // "Таня"  
alert(user.size.top); // 90
```

Компактное представление объектов

```
let user = { name: "Vasya", age: 25};
```

При создании множества объектов одного и того же вида (с одинаковыми полями) интерпретатор выносит описание полей в отдельную структуру. А сам объект остаётся в виде непрерывной области памяти с данными.

{name: "Вася", age: 25}	Вася 25
{name: "Петя", age: 22}	Петя 22
{name: "Маша", age: 19}	Маша 19

<структура: string name, number age>

Создание объектов с помощью `Object.create()`

- В ECMAScript 5 для создания объекта без собственных свойств можно использовать метод `Object.create()`. Аргумент метода – это *прототип* создаваемого объекта.
- Дополнительно `Object.create()` может принимать второй аргумент, описывающий свойства нового объекта. Об этом позже!

Создание объектов – пример

- `// o1 наследует свойства x и y из прототипа`
- `var o1 = Object.create({x:1, y:2});`
- `// o2 не наследует ни свойств, ни методов`
- `var o2 = Object.create(null);`
- `// это аналог var o3 = {};`
- `var o3 = Object.create(Object.prototype);`

Работа со свойствами и прототипы

- `var proto = { x: 10 };`
- `// создаём объект, указываем прототип`
- `var obj = Object.create(proto);`
- `alert(obj.x);` `// выводит "10"`
- `// создаём собственное свойство у obj`
- `obj.x = 100;`
- `alert(obj.x);` `// выводит "100"`
- `alert(proto.x);` `// выводит "10"`

ОБЪЕКТЫ: ПЕРЕБОР СВОЙСТВ

- циклы `for...in` – метод перебирает все перечисляемые свойства объекта и его цепочку прототипов
- `Object.keys(o)` – метод возвращает массив со всеми собственными (те, что в цепочке прототипов, не войдут в массив) именами перечисляемых свойств объекта `o`.
- `Object.getOwnPropertyNames(o)` – метод возвращает массив содержащий все имена своих свойств (перечисляемых и неперечисляемых) объекта `o`.

ОБЪЕКТЫ: ПЕРЕБОР СВОЙСТВ

цикл по свойствам `for..in` последовательно перебирает свойства объекта, имя каждого свойства будет записано в `key` и вызвано тело цикла

```
for (key in obj) {  
  /* ... делать что-то с obj[key] ... */  
}
```

Вспомогательную переменную `key` (или другое название) можно объявить прямо в цикле:

```
for (let key in menu) { // ...}
```

```
let menu = {  
  width: 300,  
  height: 200,  
  title: "Menu"  
};
```

```
for (let key in menu) {  
  // этот код будет вызван для каждого свойства объекта  
  // ..и выведет имя свойства и его значение  
  alert( "Ключ: " + key + " значение: " + menu[key] );  
}
```

Количество свойств в объекте

```
let menu = {  
  width: 300,  
  height: 200,  
  title: "Menu "  
};  
  
let counter = 0;  
for (let key in menu) {  
  counter++;  
}  
  
alert( "Всего свойств: " + counter );
```

`Object.keys(menu).length`

```
function isEmpty(obj) {  
  for (let key in obj) {  
    return false;  
  }  
  return true;  
} //нет свойств в объекте
```

Порядок перебора свойств

```
let codes = {  
  // телефонные коды в формате "код страны": "название"  
  "7": "Россия",  
  "41": "Швейцария",  
  // ..,  
  "1": "США"  
};
```

если имя свойства – число или числовая строка, то все современные браузеры сортируют такие свойства в целях внутренней оптимизации

если имя свойства – нечисловая строка, то такие ключи всегда перебираются в том же порядке, в каком присваивались

```
let user = {  
  name: "Вася",  
  surname: "Петров "  
};  
user.age = 25;  
// порядок перебора соответствует порядку присвоения свойства  
for (let prop in user) {  
  console.log ( prop );    // name, surname, age  
}
```

```
let codes = {  
  // телефонные коды в формате "код страны": "название "  
  "7": "Россия",  
  "41": "Швейцария",  
  "1": "США "  
};  
for (let code in codes)  
  console.log ( code ); // 1, 7, 41
```

```
let codes = {  
  "+7": "Россия",  
  "+41": "Швейцария",  
  "+1": "США "  
};  
  
for (let code in codes) {  
  let value = codes[code];  
  code = +code;  
  
  // ..если нам нужно именно число, преобразуем: "+7" -> 7  
  console.log ( code + ": " + value ); // 7, 41, 1 во всех браузерах  
}
```

```
"use strict";

let salaries = {
    "Вася": 100,
    "Петя": 300,
    "Даша": 250
};

let max = 0; let maxName = "";

for (let name in salaries) {
    if (max < salaries[name]) {
        max = salaries[name];
        maxName = name;    //сотр. с наибольшей зарплатой
    }
}

alert( maxName || "нет сотрудников" );
```

получает объект и умножает все численные свойства на 2

```
let menu = { width: 200, height: 300, title: "My menu"};
function isNumeric(n) {
    return !isNaN(parseFloat(n)) && isFinite(n);
}
function multiplyNumeric(obj) {
    for (let key in obj) {
        if (isNumeric(obj[key])) {
            obj[key] *= 2;
        }
    }
}

multiplyNumeric(menu);

alert( "menu width=" + menu.width + " height=" + menu.height + " title=" +
menu.title );
```

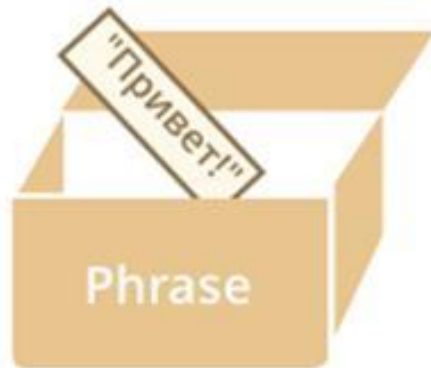
Объекты: передача по ссылке

Обычные значения: строки, числа, булевы значения, null/undefined при присваивании переменных копируются целиком («по значению»)

две полностью независимые переменные:

```
let message = "Привет";
```

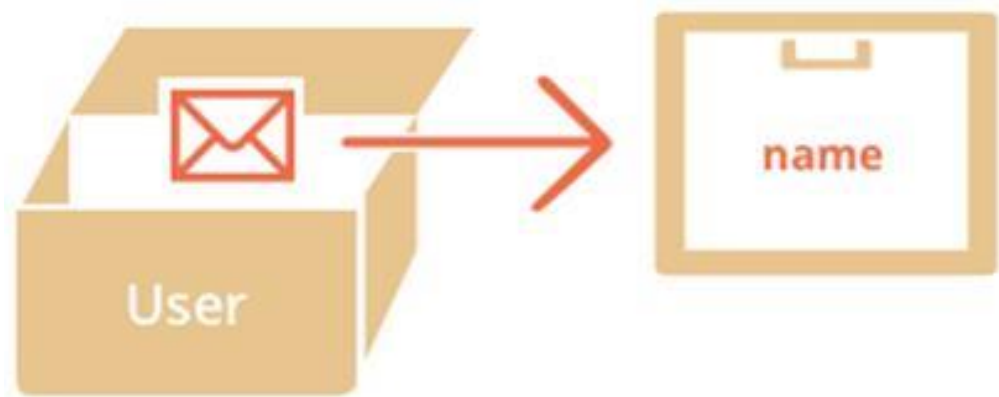
```
let phrase = message;
```



В переменной, которой присвоен объект, хранится не сам объект, а «адрес его места в памяти», иными словами – «ссылка» на него.

переменная, которой присвоен объект:

```
let user = {  
  name: "Вася"  
};
```



При копировании переменной с объектом – копируется эта ссылка, а объект по-прежнему остается в единственном экземпляре.

```
let user = { name: "Вася" }; // в переменной - ссылка
```

```
let admin = user; // скопировали ссылку
```

// две переменные, в которых находятся ссылки на один и тот же объект:



Так как объект всего один, то изменения через любую переменную видны в других переменных:

```
let user = { name: 'Вася' };
```

```
let admin = user; //копия ключа, но сейф по-прежнему один
```

```
admin.name = 'Петя'; // поменяли данные через admin
```

```
alert(user.name); // 'Петя', изменения видны в user
```

Клонирование объектов

```
let user = { name: "Вася", age: 30};  
let clone = {}; // новый пустой объект  
// скопируем в него все свойства user  
for (let key in user) {  
    clone[key] = user[key];  
}  
  
// теперь clone - полностью независимая копия  
clone.name = "Петя"; // поменяли данные в clone  
alert( user.name ); // по-прежнему "Вася"
```

Или используем: **Object.assign(dest, [src1, src2, src3...])**

Клонирование объектов

```
let user = { };
```

```
let permissions1 = { name: "John" , canView: true };
```

```
let permissions2 = { canEdit: true };
```

```
Object.assign(user, permissions1, permissions2);
```

```
console.log(user);
```

```
[object Object] {  
  canEdit: true,  
  canView: true,  
  name: "John"  
}
```

Методы объектов

Свойства-функции называют «методами» объектов.

```
let user = {  
  name: 'Василий',  
  // метод  
  sayHi: function() {  
    alert( 'Привет!' );  
  }  
};  
  
// Вызов  
user.sayHi();
```

методы объектов можно добавлять и удалять в любой момент, в том числе и явным присваиванием:

```
let user = {  
  name: 'Василий'  
};  
user.sayHi = function() {  
  // присвоили метод после создания объекта  
  alert('Привет!');  
};  
// Вызов метода:  
user.sayHi();
```

Доступ к объекту через this

Для доступа к текущему объекту из метода используется ключевое слово this

```
let user = {  
  name: 'Василий',  
  sayHi: function() {  
    alert( this.name );  
  }  
};  
user.sayHi(); // sayHi в контексте user
```

```
let user = {  
  name: 'Василий',  
  sayHi: function() {  
    alert( user.name ); // приведёт к ошибке  
  }  
};  
let admin = user;  
user = null;  
admin.sayHi();  
// !внутри sayHi обращение по старому имени, ошибка!
```

Через this метод может не только обратиться к любому свойству объекта, но и передать куда-то ссылку на сам объект целиком:

```
let user = {  
  name: 'Василий',  
  sayHi: function() {  
    showName(this); // передать текущий объект в showName  }  
};
```

```
function showName(namedObj) {  
  alert( namedObj.name );  
}
```

```
user.sayHi(); // Василий
```

Значение `this` называется *контекстом вызова* и будет определено в момент вызова функции.

```
function sayHi() { alert( this.firstName );}
```

Если одну и ту же функцию запускать в контексте разных объектов, она будет получать разный `this`:

```
let user = { firstName: "Вася" };
```

```
let admin = { firstName: "Админ" };
```

```
function func() { alert( this.firstName );}
```

```
user.f = func;
```

```
admin.g = func; // this равен объекту перед точкой:
```

```
user.f(); // Вася
```

```
admin.g(); // Админ
```

```
admin['g'](); // Админ (не важно, доступ к объекту через точку или квадратные скобки)
```

Значение this при вызове без контекста

ситуация возникает при ошибке в разработке.

//this получает значение window, глобального объекта:

```
function func() {  
    alert( this );    // выведет [object Window] или [object global]  
}  
func();
```

```
function func() {  
    "use strict";  
    alert( this );    // выведет undefined  
}  
func();
```

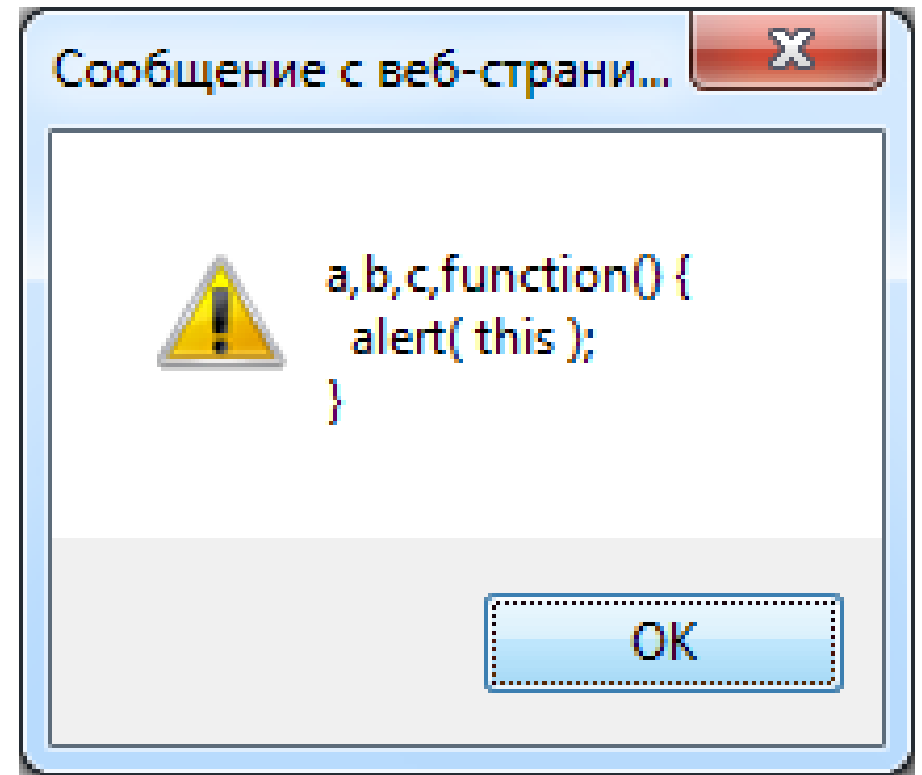
Каким будет результат?

```
let arr = ["a", "b", "c" ];
```

//Вызов в контексте массива

```
arr.push(function() {  
  alert( this );  
})
```

```
arr[3]();
```



Вызов `arr[3]()` – это обращение к методу объекта `obj[method]()`, в роли `obj` выступает `arr`, а в роли метода: `3`.

Поэтому, как это бывает при вызове функции как метода, функция `arr[3]` получит `this = arr` и выведет массив.

Каким будет результат?

```
let obj = {  
  go: function() { alert(this) }  
}  
  
(obj.go)()
```

Ошибка возникла из-за того, что после объявления obj пропущена точка с запятой.

JavaScript игнорирует перевод строки перед скобкой (obj.go)() и читает этот код как:

```
let obj = { go:... }(obj.go)()
```

Интерпретатор попытается вычислить это выражение, которое обозначает вызов объекта { go: ... } как функции с аргументом (obj.go). При этом, естественно, возникнет ошибка.

```
let name = "";  
let user = { name: "Василий",  
export: function() { return this; }  
};  
alert( user.export().name );
```

Ответ: Василий

Вызов `user.export()` использует `this`, который равен объекту до точки, то есть внутри `user.export()` строка `return this` возвращает объект `user`.

В итоге выводится свойство `name` объекта `user`, равное "Василий".

Преобразование объектов: toString и valueOf

объект преобразуется в примитив:

- Строковое преобразование – если объект выводится через alert(obj).
- Численное преобразование – при арифметических операциях, сравнении с примитивом.
- Логическое преобразование – при if(obj) и других логических операциях.

ЛОГИЧЕСКОЕ ПРЕОБРАЗОВАНИЕ

Любой объект в логическом контексте – true, даже если это пустой массив [] или объект {}.

```
if ({} && []) {  
    alert( "Все объекты - true!" ); // alert сработает  
}
```

Строковое преобразование

```
let user = {  
  firstName: 'Василий'  
};
```

```
console.log( user );
```

//выведет [object Object] - стандартное строковое

//представление пользовательского объекта

```
[object Object] {  
  firstName: "Василий"  
}
```

Если в объекте присутствует метод toString, который возвращает примитив, то он используется для преобразования.

```
let user = { firstName: 'Василий',  
  
  toString: function() {  
    return 'Пользователь ' + this.firstName;  
  }  
};  
  
alert( user ); // Пользователь Василий
```

//Результатом toString может быть любой примитив

Метод toString не обязан возвращать именно строку.

«строковое преобразование», а не «преобразование к строке»

```
let obj = {  
  toString: function() {  
    return 123;  
  }  
};  
  
alert( obj +1); // 124
```

Все объекты, включая встроенные, имеют свои реализации метода toString, например:

```
alert( [1, 2] ); // toString для массивов выводит список элементов "1,2 "
```

```
alert( new Date ); // toString для дат выводит дату в виде строки
```

```
alert( function() {} ); // toString для функции выводит её код
```

Численное преобразование

используется метод `valueOf`, а если его нет – то `toString`:

```
let room = { number: 777,  
              valueOf: function() { return this.number; },  
              toString: function() { return this.number; } };  
  
alert( +room );           // 777, вызвался valueOf  
  
delete room.valueOf; // valueOf удалён  
  
alert( +room );           // 777, вызвался toString
```

Метод `valueOf` обязан возвращать примитивное значение, иначе его результат будет проигнорирован. При этом – не обязательно числовое.

У большинства объектов нет `valueOf` (по спецификации есть, но ничего не делает, просто возвращает сам объект (не-примитивное значение!)), а потому игнорируется)

Исключением является объект Date, который поддерживает оба типа преобразований:

```
alert( new Date() ); // toString: Дата в виде читаемой строки
```

```
alert( +new Date() );
```

```
// valueOf: кол-во миллисекунд, прошедших с 01.01.1970
```

После преобразования объекта в примитив при помощи toString или valueOf, примитив может быть преобразован во что-то другое.

операция ==

```
let obj = {
```

```
  valueOf: function() {
```

```
    return 1;  };
```

```
alert( obj == true ); // true
```

```
let obj = { valueOf: function() {  
    return 1; } };  
alert( obj + "test" );  
// 1test
```

для разности объектов:

```
let a = { valueOf: function() {  
    return "1"; } };  
let b = { valueOf: function() {  
    return "2"; } };  
alert( a + b );  
// "12"  
alert( a - b );  
// "1" - "2" = -1
```

```
alert( ['x'] == 'x' ); //??
```

//результат true

Если с одной стороны – объект, а с другой – нет,
то сначала приводится объект.

У массивов нет `valueOf`, поэтому вызывается `toString`, который возвращает список элементов через запятую.

В данном случае, элемент только один – он и возвращается. Так что `['x']` становится `'x'`. Получилось `'x' == 'x'`, верно.

По той же причине верны равенства:

```
alert( ['x', 'y'] == 'x,y' ); // true
```

```
alert( [] == '' ); // true
```

Какими будут результаты alert?

```
let foo = { toString: function() { return 'foo'; },  
            valueOf: function() { return 2; } };
```

```
alert( foo );
```

```
alert( foo + 1 );
```

```
alert( foo + "3" );
```

Первый alert(foo)

Возвращает строковое представление объекта, используя toString, т.е. "foo".

Второй alert(foo + 1)

Оператор '+' преобразует объект к примитиву, используя valueOf, так что результат: 3.

Третий alert(foo + " 3 ")

То же самое, что и предыдущий случай, объект превращается в примитив 2. Затем происходит сложение 2 + '3'. Оператор '+' при сложении чего-либо со строкой приводит и второй операнд к строке, а затем применяет конкатенацию, так что результат – строка "23".

`new Date(0) - 0 = 0 // (1)`

`new Array(1)[0] + "" = "undefined" // (2)`

`({})[0] = undefined // (3)`

`[1] + 1 = "11" // (4)`

`[1,2] + [3,4] = "1,23,4" // (5)`

`[] + null + 1 = "null1" // (6)`

`[[0]][0][0] = 0 // (7)`

`({} + {}) = "[object Object][object Object]" // (8)`

`alert( [1,[5],2][1] );`

5

КОНСТРУКТОР

Конструктором становится любая функция, вызванная через new.

функции, задуманные как конструкторы, называют с большой буквы:

Animal, а не animal

```
function Animal(name) {  
  this.name = name;  
  this.canWalk = true;  
}
```

```
let animal = new Animal("ёжик");
```

```
function Animal(name) {  
  // this = {};  
  // в this пишем свойства, методы  
  this.name = name;  
  this.canWalk = true;  
  // return this;}  
}
```

технически, любая функция может быть использована как конструктор.

действия функции Конструктора:

1. Создаётся новый пустой объект.
2. Ключевое слово `this` получает ссылку на этот объект.
3. Функция выполняется. Как правило, модифицирует `this`, добавляет методы, свойства.
4. Возвращается `this`.

В результате вызова `new Animal("ёжик");` получаем такой объект:

```
animal = { name: "ёжик", canWalk: true }
```

Иногда функцию-конструктор объявляют и тут же используют:

```
let animal = new function() {  
  this.name = "Васька";  
  this.canWalk = true;  
};
```

//то же

```
let animal = { name: "Васька", canWalk: true};
```

// простая запись не подходит, когда при создании свойств объекта нужны более сложные вычисления

ПРАВИЛА ОБРАБОТКИ RETURN

Как правило, конструкторы ничего не возвращают. Их задача – записать всё, что нужно, в `this`, который автоматически станет результатом.

Но если явный вызов `return` всё же есть, то применяется простое правило:

При вызове `return` с объектом, будет возвращён он, а не `this`.

При вызове `return` с примитивным значением, оно будет отброшено.

возврат объекта:

```
function BigAnimal() {  
    this.name = "Мышь";  
    return { name: "Годзилла" }; // <-- возвратим объект  
}
```

```
alert( new BigAnimal().name );
```

```
// Годзилла, получили объект вместо this
```

```
function BigAnimal() {  
    this.name = "Мышь";  
    return "Годзилла"; // <-- возвратим примитив  
}
```

```
alert( new BigAnimal().name );  
// Мышь, получили this (а Годзилла пропал)
```

СОЗДАНИЕ МЕТОДОВ В КОНСТРУКТОРЕ

`new User(name)` создает объект с заданным значением свойства `name` и методом `sayHi`:

```
function User(name) {  
  this.name = name;  
  this.sayHi = function() { alert( "Моё имя: " + this.name ); };  
}  
  
let ivan = new User("Иван");  
  
ivan.sayHi(); // Моё имя: Иван  
  
/*ivan = {  name: "Иван",  sayHi: функция}*/
```

Локальные переменные

В функции-конструкторе бывает удобно объявить вспомогательные локальные переменные и вложенные функции, которые будут видны только внутри:

```
function User(firstName, lastName) {  
  // вспомогательная переменная  
  let phrase = "Привет";  
  
  // вспомогательная вложенная функция  
  function getFullName() {  
    return firstName + " " + lastName;  }  
  this.sayHi = function() {  
    alert( phrase + ", " + getFullName() ); // использование  
  };}  
  
let vasya = new User("Вася", "Петров");  
vasya.sayHi(); // Привет, Вася Петров
```

Возможны ли такие функции A и B в примере ниже, что соответствующие объекты a,b равны:

```
function A() { ... }
```

```
function B() { ... }
```

```
let a = new A;
```

```
let b = new B;
```

```
alert( a == b ); // true
```

Да, возможны. Они должны возвращать одинаковый объект.

При этом если функция возвращает объект, то `this` не используется.

Например, они могут вернуть один и тот же объект `obj`, определённый снаружи:

```
let obj = {};  
function A() { return obj; }  
function B() { return obj; }
```

```
let a = new A;  
let b = new B;  
alert( a == b ); // true
```

функция-конструктор Calculator, которая создает объект с тремя методами:

```
function Calculator() {  
    this.read = function() {  
        this.a = +prompt('a?', 0);  
        this.b = +prompt('b?', 0);  
    };  
    this.sum = function() {  
        return this.a + this.b;  
    };  
    this.mul = function() {  
        return this.a * this.b;  
    };  
}  
  
let calculator = new Calculator();  
calculator.read();  
alert( "Сумма=" + calculator.sum() );  
alert( "Произведение=" + calculator.mul() );
```

УКАЗАНИЕ GET/SET В ЛИТЕРАЛАХ

```
let user = {  
  name: "John",  
  surname: "Smith",  
  get fullName() {  
    return `${this.name} ${this.surname}`;  
  },  
  set fullName(value) {  
    [this.name, this.surname] = value.split(" ");  
  }  
};  
  
// set fullName запустится с данным значением  
user.fullName = "Alice Cooper";  
  
alert(user.name); // Alice  
alert(user.surname); // Cooper
```

Метод чтения объявляется с модификатором `get`, метод записи – с модификатором `set`. Имена методов рассматриваются как имя свойства.

Один из методов можно опустить (например, чтобы получить свойство только для чтения).

Свойство с методами тоже удаляется при помощи `delete`.

Атрибуты свойств

Каждое свойство объекта обладает набором *описательных атрибутов*.

эти атрибуты доступны для чтения и для установки (*тонкая настройка свойства*).

Делается такая настройка при помощи специальных методов и *дескриптора свойства*.

Список атрибутов свойства

Свойство с данными		Свойства с методами	
[[Value]]		[[Get]]	
[[Writable]]		[[Set]]	
[[Enumerable]]			
[[Configurable]]			

value – значение свойства, по умолчанию undefined

writable – значение свойства можно менять, если true. По умолчанию false.

configurable – если true, то свойство можно удалять, а также менять его в дальнейшем при помощи новых вызовов defineProperty. По умолчанию false.

enumerable – если true, то свойство просматривается в цикле for..in и методе Object.keys(). По умолчанию false.

get – функция, которая возвращает значение свойства. По умолчанию undefined.

set – функция, которая записывает значение свойства. По умолчанию undefined.

запрещено одновременно указывать значение value, writable и функции get/set

Дескриптор свойства

Дескриптор свойства – объект, обладающий свойствами из набора "value", "writable", "enumerable", "configurable" или из набора "get", "set", "enumerable", "configurable".

Дескриптор свойства можно получить у свойства или задать с его помощью атрибуты свойства.

Дескриптор свойства — получение

Получить дескриптор свойства можно при помощи метода `Object.getOwnPropertyDescriptor()`:

```
// d = {value:1, writable:true, enumerable:true, configurable:true}  
let d = Object.getOwnPropertyDescriptor({x:1}, "x");
```

```
// вернёт undefined для унаследованных и несуществующих свойств  
d = Object.getOwnPropertyDescriptor({}, "x");    // d = undefined
```

Дескриптор свойства – установка

Для создания или изменения свойства и атрибутов можно использовать метод `Object.defineProperty()`,

Аргументы:

- объект, в котором требуется выполнить изменения,
- имя создаваемого или изменяемого свойства,
- дескриптор свойства.

Дескриптор свойства – установка

```
let o = {}; // свойств нет
```

```
// создаём неперечислимое свойство x со значением 1  
Object.defineProperty(o, "x", { value : 1,  
                                writable: true,  
                                enumerable: false,  
                                configurable: true});
```

Дескриптор свойства – изменение

// сделать x доступным только для чтения

```
Object.defineProperty(o, "x", {writable: false});
```

// значение x можно изменить так:

```
Object.defineProperty(o, "x", {value: 2});
```

// сделаем x свойством с методами доступа

```
Object.defineProperty(o, "x", {get: function() {  
                                return 0;}}});
```

Дескриптор свойства

1. Если **создаём** свойство с помощью `defineProperty()` и в дескрипторе чего-то не указываем, то это будет `false` или `undefined`.
2. Если **меняем** свойство с помощью `defineProperty()` и в дескрипторе чего-то не указываем, то это соответствующий атрибут просто не меняется.
3. `defineProperty()` работает только с собственными свойствами объекта.

Настройка нескольких свойств

Если надо создать или изменить несколько свойств, можно использовать `Object.defineProperty()`.

Аргументы:

- объект, в котором требуется выполнить изменения,
- объект, отображающий имена создаваемых или модифицируемых свойств в дескрипторы этих свойств.

Настройка нескольких свойств

```
var p = Object.defineProperty({}, {  
    x: {value:1, writable:true, enumerable:true,  
configurable:true},  
    y: {value:1, writable:true, enumerable:true,  
configurable:true},  
    r: {  
        get:function(){ return  
Math.sqrt(this.x*this.x+this.y*this.y); },  
        enumerable: true,  
        configurable: true  
    }  
});
```

Настройка нескольких свойств

Настроить свойства можно и при создании объекта при помощи второго параметра функции `Object.create()`:

```
var p = Object.create(Object.prototype, {  
    x: {value:1, writable:true, enumerable:true,  
configurable:true},  
    y: {value:1, writable:true, enumerable:true,  
configurable:true},  
    r: {  
        get:function(){ return  
Math.sqrt(this.x*this.x+this.y*this.y); },  
        enumerable: true,  
        configurable: true  
    }  
});
```

Правила согласованности

Если `[[Configurable]]=false` $\Rightarrow$ нельзя изменить `[[Configurable]]` и `[[Enumerable]]`

& свойство с данными $\Rightarrow$ 1) нельзя превратить в свойство с методами 2) нельзя изменить `[[Writable]]` с `false` на `true`, но можно изменить с `true` на `false`

& свойство с данными & `[[Writable]]=false` $\Rightarrow$ нельзя изменить `value`

& свойство с методами $\Rightarrow$ нельзя изменить методы чтения и записи и нельзя превратить в свойство с данными

Правила согласованности

Дополнительно: если объект *нерасширяемый* (как это сделать – чуть позже), то можно изменить существующие собственные свойства этого объекта, но нельзя добавить в него новые свойства.

Вызовы `Object.defineProperty()` или `Object.defineProperties()` генерируют `TypeError` при нарушении правил согласованности.

Атрибуты объекта

- Атрибуты есть не только у отдельного свойства, но и у всего объекта:
- `[[Prototype]]`: ссылка на прототип объекта
- `[[Class]]`: строка, содержащая информацию о типе объекта
- `[[Extensible]]`: определяет, можно ли добавлять в объект собственные новые свойства (`true/false`)

[[Prototype]]

Устанавливается в момент создания объекта.

Метод `Object.getPrototypeOf()` возвращает прототип объекта-аргумента.

Если у объекта вызвать метод `isPrototypeOf()`, можно узнать присутствует ли объект в цепочке прототипов аргумента:

```
var isProto = p.isPrototypeOf(o);
```

[[Class]]

Устанавливается в момент создания объекта.
Стандартные конструкторы помещают туда своё имя.

```
var a = new Array(); // a.[[Class]] = "Array"
```

Но пользовательские конструкторы так делать не могут!

Прямых методов чтения нет. Но по умолчанию метод объекта `toString()` возвращает строку вида "`[object Class]`", где *Class* – это значение атрибута `[[Class]]`.

[[Extensible]]

ECMAScript 5 определяет функции для работы с этим атрибутом:

`Object.isExtensible()` – получает объект, а возвращает признак расширяемости (`true/false`);

`Object.preventExtensions()` – получает объект и делает его нерасширяемым (обратно сделать не получится).

[[Extensible]] – добавочные методы

`Object.seal()` – получает объект, делает объект нерасширяемым, и делает все свойства объекта недоступными для настройки.

`Object.freeze()` – это `seal()` + все собственные свойства с данными делаются доступными только для чтения (это не относится к свойствам с методами).

`Object.isSealed()` и `Object.isFrozen()` – проверка, что для объекта-аргумента вызывались указанные методы.

Статические методы Object

Имя метода	Описание
<code>create()</code>	Создаёт новый объект с указанными объектом прототипа и свойствами
<code>defineProperty()</code>	Добавляет к объекту именованное свойство, описываемое переданным дескриптором
<code>defineProperties()</code>	Добавляет к объекту именованные свойства, описываемые переданными дескрипторами
<code>freeze()</code>	Замораживает объект: другой код не сможет удалить или изменить никакое свойство
<code>getOwnPropertyDescriptor()</code>	Возвращает дескриптор свойства для именованного свойства объекта
<code>getOwnPropertyNames()</code>	Возвращает массив с именами всех собственных перечисляемых и неперечисляемых свойств объекта
<code>getPrototypeOf()</code>	Возвращает прототип указанного объекта
<code>isExtensible()</code>	Определяет, разрешено ли расширение объекта
<code>isFrozen()</code>	Определяет, был ли объект заморожен
<code>isSealed()</code>	Определяет, является ли объект запечатанным (<i>sealed</i>)
<code>keys()</code>	Возвращает массив, содержащий имена всех собственных перечислимых свойств переданного объекта
<code>preventExtensions()</code>	Предотвращает любое расширение объекта (но не запрещает удаление свойств)
<code>seal()</code>	Предотвращает удаление и добавление свойств объекта другим кодом

Прототипы

В JavaScript любой объект имеет атрибут `[[Prototype]]`, который хранит ссылку на другой объект или `null`.

Объект В будем называть *прототипом* объекта А, если выполняется `A. [[Prototype]] == B`.

Если же `A. [[Prototype]] == null`, будем говорить что у объекта А нет прототипа.

Наследование на основе прототипов

При **чтении** свойства объекта поиск этого свойства сначала выполняется в самом объекте, а в случае неудачи – в прототипе (и далее по цепочке прототипов).

В определённом смысле **объект наследует свойства своих прототипов.**

Классом в JavaScript называется набор объектов, имеющих общего прототипа.

Наследование на основе прототипов

Напоминание: поиск свойства в прототипе выполняется только при чтении.

Установка свойства **всегда** происходит **в самом объекте** (если нужно, свойство создаётся).

Значит, прототипы – это **не** лучший способ **разделять данные**. Они больше подходят **для разделения поведения** (т. е. методов) объектов.

Установка и чтение `[[Prototype]]`

Значение атрибута `[[Prototype]]` устанавливается в момент создания объекта

Прочитать значение атрибута `[[Prototype]]` у произвольного объекта `obj` можно при помощи метода `Object.getPrototypeOf(obj)`.

Установка прототипа – 1

Если объект создаётся при помощи литерала, его прототипом будет объект `Object.prototype`:

```
var obj = { x: 1 };  
var proto = Object.getPrototypeOf(obj);  
alert(proto == Object.prototype);    // "true"
```

Установка прототипа – 2

Если объект создаётся при помощи `Object.create()`, его прототип – это первый аргумент данного метода:

```
var p = { x: 1 };  
var o1 = Object.create(p);  
alert(Object.getPrototypeOf(o1) == p);    //  
"true"
```

```
var o2 = Object.create(null);  
alert(Object.getPrototypeOf(o2) == null); //  
"true"
```

Установка прототипа – 3

Если объект создаётся оператором `new`, прототипом будет значение свойства `prototype` конструктора.

```
var proto = { x: 1 }; // используем в качестве прототипа
```

```
function F() { };
```

```
F.prototype = proto; // установили прототип у функции
```

```
var o1 = new F, o2 = new F;
```

```
alert(Object.getPrototypeOf(o1) == proto); // "true"
```

```
alert(Object.getPrototypeOf(o2) == proto); // "true"
```

Объект `Object.prototype`

Типовые сценарии создания объекта ведут к тому, что `Object.prototype` завершает цепочку его прототипов (вызовы наподобие `Object.create(null)` не в счёт!).

Значит, у (типового) объекта будут все свойства `Object.prototype`. А добавление свойства к `Object.prototype` отразится на всех (типовых) объектах.

Свойства Object.prototype

Имя свойства	Описание
<code>constructor</code>	Конструктор объекта (по умолчанию – функция <code>Object()</code>)
<code>hasOwnProperty()</code>	Проверяет, содержит ли указанное свойство сам объект, или он унаследовал его по цепочке прототипов
<code>isPrototypeOf()</code>	Проверяет, состоит ли указанный объект в цепочке прототипов объекта, на котором был вызван данный метод
<code>propertyIsEnumerable()</code>	Проверяет, является ли указанное свойство перечисляемым
<code>toLocaleString()</code>	Вызывает <code>toString()</code>
<code>toString()</code>	Возвращает строковое представление объекта
<code>valueOf()</code>	Возвращает значение примитива указанного объекта

Объект `Object.prototype` – атрибуты

У самого объекта `Object.prototype`:

1. `[[Prototype]] = null;`
2. `[[Class]] = "Object";`
3. `[[Extensible]] = true;`
4. У всех его свойств `[[Enumerable]] = false.`

Наследование

- Пусть имеется класс Person:
- `function Person(name) {`
- `this.name = name;`
- `}`
- `Person.prototype.greet = function () {`
- `return "Hello, my name is " + this.name;`
- `};`

Наследование

- Необходимо связать класс Person с классом Developer:
- `function Developer(name, skills) {`
- `this.name = name;`
- `this.skills = skills || [];`
- `}`

Наследование

- **Первый вариант:** создать при помощи конструктора произвольный объект класса `Person` и присвоить его `Developer.prototype`.
- При этом желательно корректно установить свойство `constructor` в объекте `Developer.prototype`.
- Также `Developer.prototype` можно дополнить новыми элементами (или переопределить существующие).

Наследование

- `function Developer(name, skills) {`
- `this.name = name;`
- `this.skills = skills || [];`
- `}`

- `Developer.prototype = new Person();`
- `Developer.prototype.constructor = Developer;`
- `Developer.prototype.getSkills = function () {`
- `return this.skills.join();`
- `};`

Наследование

- `var d = new Developer("Alex", ["C#", "JS"]);`
- `alert(d.greet());` `// => "Alex"`
- `alert(d.getSkills());` `// => "C#,JS"`
- `alert(d instanceof Developer);` `// => "true"`
- `alert(d instanceof Person);` `// => "true"`

Наследование

- **Второй вариант:** вместо вызова конструктора `Person` поместить в `Developer.prototype` результат вызова `Object.create(Person.prototype)`.
- `Developer.prototype =`
`Object.create(Person.prototype);`
- `Developer.prototype.constructor = Developer;`

Наследование

- До ECMAScript 5 вместо `Object.create()` применялось решение на основе *промежуточного конструктора*:
- `var F = function () { };`
- `F.prototype = Person.prototype;`
- `Developer.prototype = new F();`
- `Developer.prototype.constructor = Developer;`

Наследование

- В конструкторе `Developer` по сути дублируется код конструктора `Person` (установка имени).
- Избежать этого можно, используя в конструкторе `Developer` косвенный вызов конструктора `Person`:
- ```
function Developer(name, skills) {
 • Person.apply(this, arguments);
 • this.skills = skills || [];
 • }
```

# Наследование

- Косвенные вызовы могут использоваться и для работы с методами прототипа базового класса:
- `Developer.prototype.toString = function () {`
- `var s = Person.prototype.greet.call(this);`
- `return s + " my skills: " + this.skills.join();`
- `};`

# \_\_proto\_\_

Объекты в JavaScript можно организовать в цепочки так, чтобы свойство, не найденное в одном объекте, автоматически искалось бы в другом.

Связующим звеном выступает специальное свойство \_\_proto\_\_.

Если один объект имеет специальную ссылку \_\_proto\_\_ на другой объект, то при чтении свойства из него, если свойство отсутствует в самом объекте, оно ищется в объекте \_\_proto\_\_.

В спецификации ECMAScript – свойство \_\_proto\_\_ обозначено как [[Prototype]].

```
let animal = {
 eats: true};
let rabbit = { jumps: true};

rabbit.__proto__ = animal;
// в rabbit можно найти оба свойства
alert(rabbit.jumps); // true
alert(rabbit.eats); // true
```

1.Первый alert выводит свойство jumps объекта rabbit.

2.Второй alert хочет вывести rabbit.eats, ищет его в самом объекте rabbit, не находит – и продолжает поиск в объекте rabbit.\_\_proto\_\_, то есть, в данном случае, в animal.

*animal*

eats: true

*\_\_proto\_\_*

*rabbit*

jumps: true



Объект, на который указывает ссылка `__proto__`, называется *«прототипом»*.

В данном случае `animal` является прототипом для `rabbit`.

Также говорят, что объект `rabbit` *«прототипно наследует»* от `animal`.

Обратим внимание – прототип используется исключительно при чтении. Запись значения, например, `rabbit.eats = value` или удаление `delete rabbit.eats` – работает напрямую с объектом.

```
let animal = { eats: true};
let rabbit = { jumps: true, eats: false};
rabbit.__proto__ = animal;
alert(rabbit.eats); // false, свойство взято из rabbit
```

**Другими словами, прототип – это «резервное хранилище свойств и методов» объекта, автоматически используемое при поиске.**

У объекта, который является \_\_proto\_\_, может быть свой \_\_proto\_\_, у того – свой, и так далее. При этом свойства будут искаться по цепочке.

# Объекты в JavaScript

1. Любой объект – это значение типа данных `object`.
2. Объект – это неупорядоченная *коллекция свойств*.
3. Каждое свойство имеет *имя* и *значение*.
4. Имена свойств – это *строки*, значения свойств могут иметь *любой тип* (число, строка, объект, функция, ...).
5. Объект позволяет *читать и записывать* значения своих свойств.

6. Объекты являются *динамическими* – обычно они позволяют добавлять и удалять свои свойства.

7. Используется *прототипная объектная модель*: любой объект может быть связан с другим объектом (*прототипом*) и иметь доступ к свойствам прототипа.

8. *Классом* называется набор объектов с общим прототипом.

9. Элементы классического ООП моделируются.

10. Объекты используют *ссылочную* семантику при присваивании.

11. Переменной с объектом можно присвоить значение `null`. Это приводит к потере ссылки на объект.

12. Движки реализуют автоматическую сборку мусора.

# Стандарт ECMAScript 2015

- Начал разрабатываться в 2011 году под именем ECMAScript 6.
- Основные цели создания:
  1. Расширить синтаксис ECMAScript.
  2. Добавить новые объекты, методы, функции.
- Выпущен 17 июня 2015 как **ECMAScript 2015** (ES2015).

# Стандарт ECMAScript 2015

- Поддержка «движками» – смотрим тут:
- <http://kangax.github.io/compat-table/es6/>
- Как использовать прямо сейчас везде:
  - a. При помощи *транспайлеров* (синтаксис)
  - b. При помощи *полифиллов* (новые функции)

# Babel (или Babel.js)

- Это самый популярный транспайлер для ES2015 (заодно выполняет функции полифилла).
- Сайт: <https://babeljs.io/>
- **(Основной) вариант использования №1:** при автоматической сборке крупных веб-проектов, как модуль сборщика для node.js.
- \*) нам не очень подходит

# Babel (или Babel.js)

- **Вариант использования №2:** трансляция кода прямо в браузере (хорошо для обучения):

<https://babeljs.io/repl/>

- **Вариант использования №3** (пока ещё работает):

- `<script src="https://js.cx/babel-core/browser.min.js"></script>`
- `<script type="text/babel">`
- `let arr = ["hello", 2];`
- `let [str, times] = arr;`
- `</script>`

# Классы в ECMAScript 2015

- В ECMAScript 2015 появился новый синтаксис для описания классов:
- `class Название {`
  - `constructor`
  - *методы*
  - `}`
- По сути, конструкция `class` — «синтаксический сахар» для задания конструктора вместе с методами прототипа.

# Класс и работа с ним

```
class Point {
 constructor(x, y) {
 this.x = x;
 this.y = y;
 }
 toString() {
 return '(' + this.x + ', ' + this.y + ')';
 }
}
```

```
let p = new Point(3, 5);
alert(p.toString());
```

Функция constructor запускается при создании new User, остальные методы записываются в User.prototype

# Во что (примерно) превращается class

```
function Point(x, y) {
 // проверка того, что вызов Point был с new
 // иначе генерируется исключение TypeError
 this.x = x;
 this.y = y;
}
```

```
Point.prototype.toString = function () {
 return '(' + this.x + ', ' + this.y + ')';
};
```

# Особенности конструкции class

- Конструктор нельзя вызвать без `new`, иначе исключение `TypeError`.
- Объявление класса ведёт себя как `let`. Оно видно только в текущем блоке и только в коде, который находится ниже объявления.
- Методы внутри `class` являются методами объекта (см. *методы объекта в литерале объекта*).
- Все методы работают в строгом режиме.
- Все методы класса не перечислимы в цикле `for/in`.

# Классы в ECMAScript 2015 – нюансы

Классы не могут содержать обычных свойств. Но в классе можно описать *методы* чтения и записи свойств.

Также для имён методов разрешено использовать выражения в квадратных скобках (как в литералах объектов).

# Классы в ECMAScript 2015 – нюансы

```
class User {
 constructor(firstName, lastName) {
 this.firstName = firstName;
 this.lastName = lastName;
 }
 get fullName() { return `${this.firstName}
${this.lastName}`; }

 ["test".toUpperCase]() { alert("PASSED!"); }
};
let u = new User('Mary', 'Mor');
alert(u.fullName);
u.TEST();
```

# Конструкция class как выражение

Конструкция `class` может использоваться как выражение. Возможное применение – передавать такое выражение в *фабрики объектов*.

```
let Point = class {
 constructor(x, y) {
 this.x = x;
 this.y = y;
 }

 toString() { return `${this.x}, ${this.y}`; }
}
```

# Статические элементы класса

Статические элементы класса объявляются с модификатором `static`.

По сути, речь идёт о добавлении свойств к объекту, описывающему функцию-конструктор.

# Статические элементы – пример

```
class Point {
 constructor(x, y) {
 this.x = x;
 this.y = y;
 }
 static createPoint(x) { return new Point(x, 0); }
 static get Zero() { return new Point(0, 0); }
 toString() { return `${this.x}, ${this.y}`; }
}
// работа со статическими элементами
alert(Point.createPoint(1).toString());
alert(Point.Zero.toString());
```

# Наследование классов

Конструкция `class` позволяет при описании класса указать **базовый класс** при помощи ключевого слова `extends`. Как и в классическом ООП, **наследник** получает доступ к методам базового класса.

В конструкторе и методах наследника доступ к предку можно получить при помощи ключевого слова `super`.

**Внимание:** в конструкторе наследника **должен** быть вызов конструктора предка через `super()`, и этот вызов должен предварять обращение к `this`.

# Наследование классов – пример

```
// это будет базовый класс
class Point {
 constructor(x, y) {
 this.x = x;
 this.y = y;
 }
 toString() {
 return `${this.x}, ${this.y}`;
 }
}
```

# Наследование классов – пример

```
// класс-наследник
class ColorPoint extends Point {
 constructor(x, y, color) {
 super(x, y);
 this.color = color;
 }
 toString() {
 return super.toString()+' in '+this.color;
 }
}
```

# Ключевое слово `super` в литералах

Конструкция `super` работает не только в классах, но и в литералах объектов (но только для *методов объектов*).

```
let animal = {
 walk() { alert("I'm walking"); }
};

let rabbit = {
 __proto__: animal,
 walk() { super.walk(); }
};
rabbit.walk();
```

# Ключевое слово `super` в литералах

Как и почему работает `super` в методах объектов?

Дело в том, что метод объект (и только он) имеет внутренний атрибут `[[HomeObject]]` – ссылка на объект, которому метод принадлежит.

Запись `super.walk()` эквивалентна

`[[HomeObject]].__proto__.walk()`

# Класс без явного конструктор

В конструкции `class` можно не задавать `constructor`. В классах без предка это эквивалентно объявлению

```
constructor() {}
```

В класса-наследниках отсутствие конструктора равносильно такому объявлению:

```
constructor(...args) {
 super(...args);
}
```

# Коллекция Map

- Map – это коллекция для хранения пар «*ключ-значение*». Ключ может быть абсолютно любым. Ключи сохраняются как есть, без преобразования типов (напомним, что в объекте ключами могут быть только строки).
- Проверка ключей на равенство почти аналогична оператору `===`, но NaN считается равным NaN.

# Свойства и методы Map

|                 |                                                                               |
|-----------------|-------------------------------------------------------------------------------|
| size            | Количество пар в экземпляре Map                                               |
| clear()         | Удаляет все пары из экземпляра Map                                            |
| delete()        | Пытается удалить пару по ключу. Возвращает <b>true</b> , если пара была в Map |
| entries()       | Итератор из массивов <i>[ключ, значение]</i> в порядке вставки пар            |
| forEach()       | Выполняет аргумент-функцию для каждой пары                                    |
| get()           | Получает элемент по ключу (нет ключа – возвращает <b>undefined</b> )          |
| has()           | Проверяет, есть ли ключ в экземпляре Map                                      |
| keys()          | Итератор ключей в порядке вставки пар                                         |
| set()           | Устанавливает значение для ключа и возвращает экземпляр Map                   |
| values()        | Итератор значений в порядке вставки пар                                       |
| <i>Итератор</i> | Итератор из массивов <i>[ключ, значение]</i> в порядке вставки пар            |

# Коллекция Map – пример 1

- `let map = new Map();`
- `map.set("1", "str");` // ключ - строка
- `map.set(1, "num");` // ключ - число
- `map.set(true, "bool");` // ключ - булево  
значение
- `alert(map.size);` // 3 – количество пар
- // в обычном объекте это было бы одно и то же
- `alert(map.get(1));` // num
- `alert(map.get("1"));` // str

# Коллекция Map – пример 2

- `// аргументом конструктора может быть итерируемый набор пар`
- `let map = new Map([["1", "str"], [1, "num"]]);`
- `// NaN как ключ`
- `map.set(NaN, "Not a Number");`
- `// цепочка set()`
- `map.set(1, "one").set(2, "two");`
- `// перебор пар`
- `for(let [key, value] of map) alert(key + " = " + value);`
- `// перебор пар (альтернатива)`
- `for(let [key, value] of map.entries()) alert(key + " = " + value);`
- `// перебор ключей`
- `for (let key of map.keys()) alert(key);`
- `// перебор значений`
- `for (let value of map.values()) alert(value);`
- `map.forEach((value, key, map) => alert(key + " = " + value));`

# Коллекция Set

- Set – множество неповторяющихся значений.
- Коллекция Set имеет схожесть с Map: ключом может быть абсолютно произвольное значение, которое сохраняется без преобразования типов.
- Проверка ключей на равенство в Set осуществляется аналогично проверке в Map (NaN считается равно NaN).

# Свойства и методы Set

|                        |                                                                                  |
|------------------------|----------------------------------------------------------------------------------|
| <code>size</code>      | Количество значений в экземпляре Set                                             |
| <code>add()</code>     | Добавляет значение в Set (если его там нет), возвращает экземпляр Set            |
| <code>clear()</code>   | Удаляет все значения из экземпляра Set                                           |
| <code>delete()</code>  | Удаляет указанное значение. Возвращает <code>true</code> , если было что удалять |
| <code>entries()</code> | Итератор из массивов [ <i>значение, значение</i> ] в порядке вставки значений    |
| <code>forEach()</code> | Выполняет аргумент-функцию для каждого значения                                  |
| <code>has()</code>     | Проверяет, есть ли значение в экземпляре Set                                     |
| <code>keys()</code>    | Итератор <b>значений</b> в порядке вставки (то же, что и <code>values()</code> ) |
| <code>values()</code>  | Итератор значений в порядке вставки                                              |
| <i>Итератор</i>        | Итератор значений в порядке вставки                                              |

# Коллекция Set – пример

- `// конструктору Set можно передать итерируемый объект`
- `let set = new Set([1, "2", false]);`
- `set.add(NaN).add(1);`
- `// 1, 2, false, NaN`
- `set.forEach((value, key, set) => alert(key));`
- `set.delete(false);`
- `// 1, 2, NaN`
- `for (let value of set.values()) alert(value);`

# Коллекции WeakMap и WeakSet

- WeakSet – особый Set, не препятствующий сборщику мусора удалять свои элементы. То есть, если некий объект присутствует **только** в WeakSet, он удаляется из памяти.
- Аналогично ведёт себя WeakMap для Map.
- **Важно:** ключи в WeakMap и значения в WeakSet – только объекты (иначе TypeError).

# Применение WeakSet/WeakMap

- У нас есть *элементы* на странице. Мы для элементов хотим хранить *обработчики* событий.
- Поместим пару *элемент-обработчик* в WeakMap (элемент будет ключом).
- Если удалить элемент на странице, сборщик мусора автоматически удалит и обработчик для элемента из WeakMap. Таким образом, WeakMap избавляет нас от необходимости вручную удалять вспомогательные данные, когда удалён основной объект.

# Ограничения WeakSet/WeakMap

- Нет свойства `size`
  - Нельзя перебрать элементы (ключи, значения)
  - Нет метода `forEach()`
  - Нет метода `clear()`
- 
- Ограничения связаны с тем, что с `Weak`-коллекциями в **независимом потоке** работает сборщик мусора.

# Модули

- *Модулем* считается файл с JS-кодом. В этом файле ключевым словом **export** помечаются переменные и функции, которые могут быть использованы снаружи.
- Другие модули могут подключать их через вызов **import**.
- \*) Известные системы организации модулей: [AMD](#), [CommonJS](#), [UMD](#). Они требуют различных библиотек или систем сборки для использования.

# Экспортирование

- Ключевое слово `export` указывается:
  - перед объявлением переменных, функций и классов;
  - отдельно, при этом в фигурных скобках указывается, что именно экспортируется.
- В последнем случае при помощи `as` можно указать *экспортируемое (внешнее) имя* элемента.
- **Внимание:** экспортируемый элемент должен иметь имя (анонимные функции экспортировать нельзя).

# Экспортирование – пример

- `// экспорт при объявлении`
- `export let one = 1;`
  
- `// отдельный экспорт + новое имя`
- `let two = 2;`
- `export {two as twice};`
  
- `// экспорт класса`
- `export class User {`
- `constructor(name) {`
- `this.name = name;`
- `}`
- `}`

# Импортирование

- Код подключает экспортированные значения из модуля при помощи ключевого слова `import`:
- `import {one, twice} from "md";`
- Здесь `"md"` – это модуль (файл `md.js`); `one`, `twice` – импортируемые переменные.
- В результате импорта появятся локальные переменные `one` и `twice` со значениями из модуля (на момент экспорта).

# Импортирование

- При импорте можно изменить имя при помощи as:
- `import {one as it1, twice as it2} from "md";`
- Можно импортировать все значения сразу в виде объекта вызовом `import * as obj:`
- `import * as numbers from "md";`

# Экспорт элемента по умолчанию

- Для одного из экспортируемых элементов можно использовать сочетание `export default`:
- `export default class User { . . . }`
- При импорте такого элемента не нужно использовать фигурные скобки:
- `import User from "md";`

# Модули и системы сборки

- Важно: ECMAScript 2015 описывает, как импортировать и экспортировать значения из модулей, но он ничего не говорит о том, как эти модули искать, и загружать.
- Поэтому **модули реально работают только в составе систем сборки!** Например, Babel в REPL для импорта модулей генерирует вызов функции `require()` – стандартная функция Node.js для загрузки модулей.

# Прокси

- *Прокси* – особый объект, который может перехватывать обращения к другому объекту и модифицировать их.
- Синтаксис создания прокси:
- `let proxy = new Proxy(target, handler);`
  - `target` – обращения к этому объекту перехватываем;
  - `handler` – объект с «ловушками»: функциями-перехватчиками для операций с `target`.

# Ловушки

- *Ловушки* (traps) – функции со специальными именами. Например, ловушка `get()` срабатывает при чтении свойства из `target`.
- Аргументы ловушки `get()`:
  - `target` – целевой объект (первый аргумент `new Proxy()`)
  - `property` – имя свойства.
  - `receiver` – прокси либо его наследник. Этот аргумент используется редко.

# Пример ловушки get()

- `let user = { name: "Alex" };`
- `let receiver = new Proxy(user, {`
  - `get(target, property, receiver) {`
    - `alert(target[property]);`
    - `return target[property].toUpperCase();`
    - `}`
  - `});`
- `// выведет "Alex" при помощи alert()`
- `// x = "ALEX"`
- `let x = receiver.name;`

# Известные ловушки (почти все)

| Ловушка                       | Когда срабатывает                                 | Что должна возвращать                      |
|-------------------------------|---------------------------------------------------|--------------------------------------------|
| <code>apply()</code>          | При вызове функции (но без <code>new</code> )     | Результат вызова функции                   |
| <code>construct()</code>      | Вызов функции, используя <code>new</code>         | Результат вызова (объект)                  |
| <code>defineProperty()</code> | Вызов метода <code>defineProperty()</code>        | <code>true</code> , если создала свойство  |
| <code>deleteProperty()</code> | При удалении свойства                             | <code>true</code> , если удаление успешно  |
| <code>get()</code>            | При чтении свойства                               | Значение свойства                          |
| <code>getPrototypeOf()</code> | При чтении прототипа                              | Прототип                                   |
| <code>has()</code>            | При проверке наличия свойства ( <code>in</code> ) | <code>true</code> , если свойство есть     |
| <code>set()</code>            | При записи в свойство                             | <code>true</code> , если запись состоялась |
| <code>setPrototypeOf()</code> | При записи прототипа                              | Ничего                                     |